

08内排序

1请模拟快速排序的分割函数，给出下面序列的第一轮排序过程。要求写出每一次交换后的序列状态（当前 pivot 为

末尾元素，从小到大排序)

第八章

1. left

left

50 pivot

piva
right

20. 89, 63, 39, 87, 92, 22, 87', 12, 6, 70', 94, 17, 31, 11, 50

↓ ↓ ↓
— 89, 63, 39, 81, 92, 22, 87, 12, 6, 70, 94, 17, 31, 11, 70

11 $\downarrow$ 89 63, 39, 81, 92, 22, 87, 12, 6, 70, 94, 17, 31 $\downarrow$ 70

11 $\downarrow$ 63, 39, 87, 92, 22, 87', 12, 6, 70', 94, 11, 31, 89, 70

1) 31 ¹63, 39 81, 92, 22, 81', 12, 6, 70', 94, 17, ~~34~~ [↓]89, 70

11 31 — 39.87.92, 22.87', 12, 6, 70', 94, 17, 63.89, 70

11 31 17 39 87 92, 22, 87', 12, 6, 70', 94 — 63, 89, 70

11 31 17 39 $\downarrow$ 92, 22, 87', 12, 6, 70', 94 87, 63, 89, 70

1131 1739 6 $\overset{\downarrow}{9} 2.22, 87', 12, \overset{\downarrow}{-} 75', 94.87, 63.89.70$

11 31 17 39 6 — 22, 87, 12, 92, 70, 94, 87, 63, 81, 70

$11, 31, 17, 39, 6, 12, 22, 81, \underline{\quad}, 92, 70, 14, 81, 0, 0, 0$

11, 31, 17, 39, 6, 12, 22 $\downarrow$ 87', 92, 70', 94, 87, 63, 89, 70

11, 31, 17, 39, 6, 12, 22, 50 87, 92, 70, 74, 81, 85, 117

2. MS17

$$\frac{0}{6}$$
$$\frac{1}{12, 17, 11}$$
$$\frac{2}{22} \quad \frac{3}{39,31}$$

4

5

L

7

50

63

0.75'

89.87.87

9

92,94

2桶排序

0 1桶中 2 3桶中 5/6 7桶中不变, 8桶中 9桶中
 6 | 11, 12, 17 | 22 | 31, 39 | 50 | 63 | 87, 87' | 89 | 92, 94

LSD

0 1 2 3 4 5 6 7
 70, 70', 50 31, 11 92, 22, 12 63 94 6 87, 87', 17

8 9
 89, 39

70, 70', 50, 31, 11, 92, 22, 12, 63, 94, 6, 87, 87', 17, 89, 89

用次低位排序, 桶内

0 1 2 2 3 5 6 7 8
 6 11, 12, 17 22 31, 39 50 63 70, 70' 87, 87', 89

9
 92, 94

31
 6, 11, 12, 17, 22, 31, 50, 63, 70, 70', 87, 87', 89, 92, 94

3.

3

给定正整数 和一个含有 n 个数的集合 $A=\{x_1, \dots, x_n\}$ ，对于 A 的所有子集，共有 个求和结果。请从集合 中得到这些结果并排序输出，给出算法过程和时间复杂度。

算法过程如下

1. 计算总和

- 计算集合 A 中所有元素的和 S ，即 $S = \sum x_i (i = 1 \text{ 到 } n)$
- 这将确定可能子集和的范围（从 0 到 S ）

2. 初始化动态规划数组

- 创建一个布尔数组 dp ，大小为 $S+1$ （索引从 0 到 S ）
- 初始时所有元素设为 `false`
- 设置 $dp[0] = \text{true}$ ，表示空子集的和为 0

3. 更新动态规划数组

- 对于集合 A 中的每个元素 x_i ：
 - 从 $j = S$ 递减到 x_i ：
 - 如果 $dp[j - x_i]$ 为 `true`，则设置 $dp[j] = \text{true}$
- 这表示如果和 $j - x_i$ 可以被某个子集达到，那么和 j 也可以被包含 x_i 的子集达到

4. 输出排序结果

- 遍历 dp 数组从索引 0 到 S
- 如果 $dp[j]$ 为 `true`，则输出 j
- 由于索引是递增的，输出自然按升序排列

时间复杂度分析

- 计算总和 S 的时间： $O(n)$
- 初始化 dp 数组的时间： $O(S)$
- 更新 dp 数组的时间： $O(n \times S)$
- 输出结果的时间： $O(S)$

总时间复杂度： $O(n \times S)$

其中 S 是集合 A 中所有元素的和。需要注意的是， S 可能很大，尤其是在元素值较大的情况下，那样的话可能效果不佳，但该算法在 n 较小或 S 合理时是高效的。